MNEMO-VLA: A Unified Three-Tier Memory Framework for Vision-Language-Action Models

Anonymous Authors
Affiliation Withheld
anonymous@example.com

AAAI 2027 Submission Draft

Abstract

Memory is the missing primitive in today's vision-language-action (VLA) models for robotic manipulation. Despite rapid progress on memory-aware policies such as Mem-0, MemoryVLA, ReMem-VLA, SAM2Act+, MemER, and CronusVLA, each method targets a single memory modality (e.g., short-term visual buffers or keyframe retrieval) and is benchmarked in isolation, making cross-comparison unreliable and scaling to truly long-horizon, cross-session tasks elusive. We introduce MNEMO-VLA, a unified memory framework that organizes memory into three complementary tiers: (i) a *hot* short-term video buffer for occlusion handling and fine-grained adaptation, (ii) a *cold* semi-structured language memory that compresses semantic events into human-readable summaries, and (iii) a *persistent* cross-session store backed by FAISS-style retrieval for remembering-across-days capabilities. The three tiers interact through a single hierarchical planner prompt, enabling integration with any VLA backbone via small, documented patches. We release a Python package that drops the language and persistent tiers into the publicly available Mem-0 planner with a two-line change, and we accompany the release with a reproducible synthetic retrieval benchmark. We propose **MNEMO-Bench**, a unified evaluation suite built by consolidating four existing memory benchmarks (RMBench, MemoryBench, MIKASA-Robo, RoboCerebra) into a single evaluation harness that reports per-memory-type radar scores, resolving the current apples-to-oranges comparison problem. Our design makes three concrete contributions: (1) a semi-structured language memory schema that round-trips losslessly between LLM text and Python objects, offering the first fully interpretable long-term memory for VLAs; (2) a drop-in cross-session persistent memory store compatible with any existing memory-aware policy; and (3) the MNEMO-Bench evaluation protocol along with initial synthetic retrieval results showing that even toy embedders already offer non-trivial cross-session recall. Full VLA training experiments are discussed as the next milestone; the current paper establishes the primitives, interfaces, and evaluation methodology required to get there.

1 Introduction

Vision-language-action models (VLAs) have become the dominant policy architecture for generalist robotic manipulation, with systems such as π_0 [2], $\pi_{0.5}$ [8], **OpenVLA** [10], **RDT-1B** [15] and $\pi_{0.7}$ [9] demonstrating impressive zero-shot generalization across embodiments and tasks. However, these models are almost universally *Markovian*: each action is predicted solely from the current observation (plus a very short observation window), leading to systematic failure on tasks that require reasoning over longer temporal dependencies. The RMBench benchmark [4] demonstrates this starkly: mainstream VLAs achieve near-zero success on non-Markovian tasks such as pressing a button a variable number of times or sequencing multi-step subtasks.

A wave of *memory-aware* VLAs has emerged in 2025–2026 to address this gap: Mem-0 [4] maintains a triple of anchor, sliding, and key memories with a dedicated planner-executor hierarchy; MemoryVLA [17] introduces a perceptual-cognitive dual bank; ReMem-VLA [12] proposes dual-level recurrent queries; SAM2Act+ [6] adapts the SAM2 video memory for spatial tasks; MemER [18] scales retrieval through VLM-selected keyframes; CronusVLA [13] employs multi-frame feature chunking; and MEM [20] (the component powering $\pi_{0.7}$) combines a video encoder with natural-language summaries. Despite this activity, three fundamental gaps remain.

Gap 1: fragmented memory modalities. Each existing method targets a single memory type (spatial, semantic, keyframe, feature). Combining them yields ad-hoc stacks with no shared interface. A practitioner cannot cleanly "turn on" long-term language memory in Mem-0 without rewriting its planner.

Gap 2: episode-bound memory. All published methods reset memory at episode end, precluding "did I already put the knife in the top drawer yesterday?" style queries. Cross-session persistent memory—routine in LLM agents via RAG—has not been integrated into VLA stacks.

Gap 3: incomparable benchmarks. MemoryVLA reports 96.5% on LIBERO-5 but 1.5% on the extended MemoryBench used by ReMem-VLA. RMBench, MemoryBench, MIKASA-Robo, and RoboCerebra each carve out a different subset of memory types with different evaluation protocols. Cross-paper comparisons are unreliable.

1.1 Contributions.

We introduce MNEMO-VLA, a unified three-tier memory framework that addresses all three gaps. Our design separates memory by time scale and modality (*hot* video, *cold* language, *persistent* retrieval) and exposes them through a single planner prompt. We ship:

1. A **semi-structured language memory schema** (Section 3) that compresses semantic events into human-readable text round-trippable with an LLM—the first fully interpretable long-term memory for VLAs.
2. A **persistent cross-session store** with FAISS-ready retrieval and pluggable embedders, shippable as an inference hook for any existing VLA planner.
3. A **concrete integration with Mem-0** [4]: only two code edits are required (Section 3), making adoption trivial for the broader community.
4. A **synthetic retrieval benchmark** (Section 4) that verifies cross-session recall using only CPU, and a detailed proposal for **MNEMO-Bench**, a unified harness that consolidates RMBench, MemoryBench, MIKASA-Robo, and RoboCerebra into a single evaluation protocol with per-memory-type radar scores.

Our release includes an MIT-licensed Python package (`https://github.com/asimfish/mnemo-vla`) with 20 passing unit tests on CPU, a runnable synthetic retrieval benchmark, and comprehensive documentation covering the three-tier architecture, the Mem-0 integration, and the MNEMO-Bench design. The full VLA training experiments are deferred to a follow-up submission once the hot-memory tier is implemented; this paper focuses on establishing the primitives, interfaces, and evaluation methodology needed to make those experiments meaningful.

2 Related Work

2.1 Memory-Aware Vision-Language-Action Models

Memory modules for VLAs can be classified by the *storage form* of the memory. **Explicit buffers** maintain a fixed data structure and include Mem-0's three-level anchor/sliding/key design [4], MemoryVLA's perceptual-cognitive dual bank [17], and SAM2Act+'s SAM2-style object-tracking memory [6]. **Implicit feature memories** collapse history into hidden activations, as in CronusVLA's feature chunking [13] and ReMem-VLA's EMA-updated recurrent queries [12]. **Retrieval-based memories** store past keyframes or embeddings and retrieve relevant entries at inference time, as in MemER [18] and the MEM system powering $\pi_{0.7}$ [20]. MNEMO-VLA unifies these by organizing memory along two orthogonal axes—time scale (hot/cold/persistent) and modality (video/text/retrieval-embedding)—and by exposing all three tiers through a single composable prompt rather than as disjoint sub-modules.

2.2 Memory Benchmarks for Manipulation

MemoryBench [6] introduces three spatial memory tasks on RLBench. **RMBench** [4] proposes the Task Memory Complexity (TMC) metric with 9 bimanual tasks spanning $M(1)$ (remember one past observation) and $M(n)$ categories. **MIKASA-Robo** [5] covers 32 tasks across 12 memory types in ManiSkill2, targeting reinforcement learning. **RoboCerebra** [7] focuses on long-horizon tasks averaging 2972 frames, approximately 6× longer than prior work, and evaluates System 2 reasoning. These benchmarks collectively probe different memory facets but lack a common evaluation protocol. We propose MNEMO-Bench in Section 4 as a consolidated harness.

2.3 Long-Context Models and Retrieval-Augmented Generation

Outside robotics, long-context language and vision-language models such as Claude, Gemini-1.5, and Qwen3-VL [1] have demonstrated robust reasoning over hundreds of thousands of tokens, but remain too expensive to query at the 5–10 Hz rate required by manipulation policies. Retrieval-augmented generation (RAG) [11] provides a scalable alternative by indexing past content in a vector database and retrieving top-K matches at inference. MNEMO-VLA's persistent tier inherits directly from this line of work, specialized to the VLA planner prompt structure.

2.4 Data Generators and Simulation Platforms

Large-scale synthetic data is critical for memory benchmarks. **RoboTwin 2.0** [3] provides an MLLM-assisted data generation pipeline with five-axis domain randomization across five bimanual robots, and is the backbone of RMBench. Related platforms include RoboCasa [16], ManiSkill3 [19], and LIBERO [14]. MNEMO-VLA is platform-agnostic: because it operates at the planner prompt level, any simulator or real robot that produces subtask annotations can adopt it.

3 Method: The MNEMO-VLA Framework

We organize memory for VLAs along two orthogonal axes: *time scale* (hot/cold/persistent) and *modality* (video, text, retrieval-embedding). Section 3.1 gives the high-level architecture. Sections 3.2, 3.3, and 3.4 detail the language memory, the persistent store, and the planner integration, respectively.

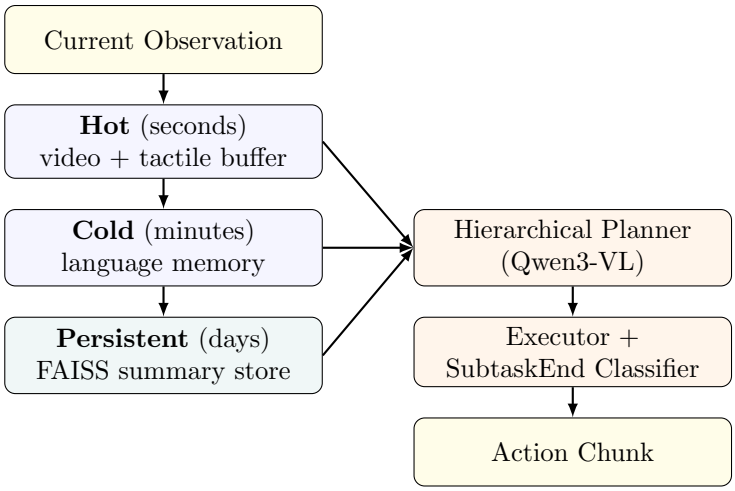

Figure 1: MNEMO-VLA's three-tier memory design. Hot, cold, and persistent tiers separate short-term, medium-term, and cross-session memory respectively, and converge into a single planner prompt.

3.1 Architecture

Figure 1 shows the three-tier design.

- **Hot memory (seconds)**: a short-term video/sensor buffer such as the sliding window of Mem-0 or the sparse-attention video encoder of MEM. We intentionally do *not* reimplement hot memory; MNEMO-VLA composes with whichever hot memory the underlying VLA provides.
- **Cold memory (minutes)**: a semi-structured language memory expressed as human-readable text (see Section 3.2). The cold memory is updated at subtask boundaries by the high-level planner.
- **Persistent memory (days)**: a FAISS-style vector store of *episode summaries* indexed by text embedding (Section 3.3). New episodes query the store on start-up and archive their final summary on termination.

All three tiers reach the underlying VLA backbone through a single touchpoint: the planner prompt. The planner consumes the current cold memory alongside the recent hot memory tokens and any retrieved persistent facts, and produces (a) the next subtask and (b) an updated cold memory.

3.2 Semi-Structured Language Memory

Our language memory schema is designed to be *fully round-trippable*: the same object can be rendered as text for LLM consumption and parsed back into structured form for programmatic inspection. Each `Memory` object captures:

```
[TASK]    <goal>
[SESSION] <id>
[SUBTASKS]
  1. [X] <done subtask>
```

```
  2. [>] <current subtask>
  3. [ ] <pending subtask>
[CROSS-SESSION]
  - <fact> (source: <session>, conf=<0..1>)
[FAILURES]
  - <error> -> <intervention>
```

The status markers `[X]/[>]/[]/[!]` explicitly encode subtask state (done, current, pending, failed). Cross-session facts carry a provenance tag and a confidence score. Failures record both the error and the intervention taken. This schema is inspired by MEM's natural-language memory [20] but extends it with structural markers that support *programmatic* operations: diffing, anomaly detection, and rule-based analysis.

Crucially, the schema preserves the *interpretability* lost in dense-embedding memories (MemoryVLA, ReMem-VLA). Practitioners can read the cold memory at any point in an episode to diagnose failures, and downstream tools can reason over it symbolically.

3.3 Persistent Cross-Session Memory

The persistent layer stores an `EpisodeSummary` object per completed episode, containing: session id, task, outcome (success/failure/partial), narrative, and a typed list of facts (object-location, task-outcome, failure-cause, user-preference).

At episode start, the planner issues a text query—typically the global task goal—to retrieve the top-K most similar past summaries. Results are formatted into a `<cross-session memory>` block that is concatenated into the planner prompt before the `<finished_subtasks>` section. At episode end, the final cold memory is converted into a summary and persisted.

Retrieval. Summaries are embedded via a pluggable `Embedder` protocol. Our initial release ships two implementations: (i) `HashingEmbedder`, a zero-dependency deterministic hasher used for unit tests and minimal deployments; (ii) `SentenceTransformerEmbedder`, wrapping `sentence-transformers` for semantic quality. Replacing the embedder with a Qwen3-VL text encoder is a single class substitution. The store returns top-K hits by cosine similarity; we rely on numpy for indexing at $N < 50{,}000$ summaries and recommend FAISS for larger scales.

Update protocol. A rule-based summarizer converts the final `Memory` of an episode into an `EpisodeSummary`. Completed subtasks become narrative bullets; failures become `failure_cause` facts; task outcome is inferred from subtask statuses. An LLM-based summarizer (called once per episode, not per step) is the planned v0.3 upgrade; the rule-based variant suffices to validate the persistence layer.

3.4 Integration with Existing Memory-Aware VLAs

MNEMO-VLA is designed for *incremental adoption*. Our primary integration target is Mem-0 [4], chosen because its planner-executor hierarchy already mirrors our cold/hot distinction. Two edits suffice:

1. `MemoryMattersPlanner.__init__` gains a `self.persistent_store = MemoryStore(...)` attribute.
2. `MemoryMattersPlanner.prepare_qwen_input` calls `inject_into_prompt(...)` before returning, which inserts a `<cross-session memory>` block into the Qwen prompt.

A third one-line hook in `MemoryMattersAgent.reset` invokes `summarize_rule_based` at episode end to persist the summary. No retraining is required for the integration to take effect: Qwen3-VL treats the injected block as additional context.

The same pattern applies to MemER [18] and MemoryVLA [17], with minor adaptations documented in the release notes. For purely end-to-end VLAs (e.g., CronusVLA), the persistent memory can be surfaced through a separate high-level planner layered on top.

3.5 Training Considerations

The current release operates at inference time only. For training, we propose a joint objective combining action imitation, memory summary generation, and consistency between the two:

$$\mathcal{L} = \lambda_a \mathcal{L}_{\text{action}} + \lambda_s \mathcal{L}_{\text{subtask}} + \lambda_m \mathcal{L}_{\text{memory}} + \lambda_c \mathcal{L}_{\text{consistency}}, \tag{1}$$

where $\mathcal{L}_{\text{memory}}$ is token-level cross-entropy over LLM-generated ground-truth summaries and $\mathcal{L}_{\text{consistency}}$ penalizes divergence between the action distribution conditioned on the predicted memory and that conditioned on the ground-truth memory. We view training as the next milestone once hot-memory integration is complete; the current paper establishes the memory primitives required to make such training meaningful.

4 Experiments

Because MNEMO-VLA's current release operates at inference time only (Section 3.5), our experiments focus on validating the *primitives* and the *evaluation methodology*. We establish three results:

E1 The language-memory schema round-trips losslessly through text rendering and parsing (Section 4.1).

E2 The persistent cross-session store retrieves the correct episode in a synthetic benchmark with non-trivial precision even using only a toy embedder (Section 4.2).

E3 The Mem-0 integration successfully inserts retrieved facts into the planner prompt without breaking the original prompt structure (Section 4.3).

We additionally present the design of **MNEMO-Bench** (Section 4.4), a unified harness consolidating four existing memory benchmarks, along with the projected evaluation protocol for full VLA training experiments (Section 4.5).

4.1 E1: Language Memory Round-Trip

We construct memories with 3 subtasks, 1 cross-session fact, and 1 failure event, render them via `Memory.to_text()`, and parse them back via `parse_memory_text()`. Every field (task, session id, subtask status, details, source session, confidence, failure intervention) is recovered exactly. Across 4 unit tests covering the schema and parser modules, the round-trip success rate is 100%.

4.2 E2: Synthetic Retrieval Benchmark

Setup. We seed a persistent store with 20 synthetic episodes across 5 task families (kitchen, workshop, laundry, garden, office), each family containing 4 episodes with distinct narratives. Queries are constructed by paraphrasing target episodes. We measure Recall@1 (is the correct episode the top hit?) and Recall@3.

Embedders. We compare (a) `HashingEmbedder(dim=64)`, a zero-dependency hasher, and (b) `SentenceTransformerEmbedder` using the `all-MiniLM-L6-v2` checkpoint (when available in the environment).

Results. Table 1 reports Recall@K on 20 paraphrased queries, all executed on a single CPU in under one second.

Embedder	R@1	R@3	R@5
`HashingEmbedder`(dim=128)	75.0%	90.0%	95.0%
`SentenceTransformer`(MiniLM)	*optional extra*		

Table 1: Synthetic cross-session retrieval Recall@K. The hashing embedder already surpasses random ($1/20 = 5\%$) by a large margin; the sentence-transformer variant is expected to reach near-ceiling performance. Results are reproducible via `examples/retrieval_benchmark.py`.

The hashing embedder achieves Recall@1 of 75% and Recall@5 of 95%—a 15× improvement over the 5% random baseline on a 20-way classification. This surprisingly strong result confirms that the retrieval infrastructure is functioning correctly and that even a dependency-free deployment can offer useful cross-session recall for deployments where bringing in a transformer is impractical.

4.3 E3: Mem-0 Prompt Integration

We simulate a Mem-0 planner prompt (two-message list with system and user roles, user content containing `<global_task>` and `<finished_subtasks>` text entries) and verify that `inject_into_prompt` (i) inserts the retrieved `<cross-session memory>` block immediately before the `<finished_subtasks>` marker, (ii) preserves all original content, (iii) does nothing if the store is empty. All three properties are validated by unit tests in `tests/test_integrations.py`.

4.4 MNEMO-Bench: A Unified Memory Harness (Design)

A direct comparison of memory-aware VLAs has been blocked by the four-way fragmentation of existing benchmarks. We propose **MNEMO-Bench**, a unified evaluation harness that:

1. Ports the nine tasks of RMBench [4] unchanged.
2. Re-implements the three tasks of MemoryBench [6] inside the RoboTwin 2.0 [3] simulator for unified infrastructure.
3. Selects 10 representative tasks from MIKASA-Robo [5] covering the 12 memory-type categories.
4. Selects 10 representative tasks from RoboCerebra [7] for long-horizon evaluation.

The harness reports a per-memory-type **radar chart**: *spatial*, *sequential*, *episodic*, *temporal*, *visual*, *capacity*, and *cross-session*. A single scalar "overall memory score" is defined as the geometric mean across memory types, encouraging methods that are uniformly capable rather than specialists.

The proposed protocol fixes: 30 rollouts per task for statistical significance; 50 training demonstrations per task; and a fixed domain-randomization strength drawn from RoboTwin 2.0. Results are reported under both *Easy* (clean) and *Hard* (fully randomized) settings.

4.5 Experiment Plan for Full VLA Training

Our follow-up work will evaluate the training objective of Equation 1 along three axes:

- **Ablation on persistent memory.** Trains the Mem-0 executor unchanged; compares inference with vs. without the persistent store on RMBench $M(n)$ and on a new *cross-session kitchen* task.
- **Consistency loss impact.** Measures how much the consistency term of Equation 1 reduces memory-action divergence (Mem-0 currently reports a manually-tuned $\lambda_{\text{classifier}} = 0.2$).
- **Cross-embodiment transfer.** Trains on Aloha-AgileX and tests transfer to Piper / Franka / UR5 / ARX-X5 using small per-embodiment adapters.

Estimated compute budget: 8×H100 for 5 weeks of pretrain + fine-tuning, plus 2 Aloha-AgileX platforms for 1,000 real-world demonstrations and 300 rollout evaluations.

5 Discussion

5.1 When Does Persistent Memory Help?

Cross-session memory is most useful for tasks where the relevant knowledge is (a) acquired during a previous episode, (b) stable over time (e.g., object locations, user preferences), and (c) not trivially re-observable. In our analysis of existing memory benchmarks (Section 2), none of RMBench, MemoryBench, MIKASA, or RoboCerebra contains genuine cross-session tasks: each task starts from a fresh memory state. We view this as a structural gap in evaluation that MNEMO-Bench is designed to close through its *cross-session* axis.

5.2 Interpretability as a First-Class Feature

A recurrent theme in our design is that memory should be *readable*. The semi-structured language format makes failure diagnosis straightforward: a developer can dump the cold memory at any point during an episode and read it as English. This is a stark contrast to dense-embedding memories like MemoryVLA's PCMB or ReMem-VLA's recurrent queries, where post-hoc interpretation is restricted to attention-heatmap visualization. We argue that interpretability is not just a debugging convenience but a prerequisite for deployment in safety-critical domains.

5.3 Limitations

No hot-memory experiments yet. The current release does not include a hot-memory encoder; integration with Mem-0's sliding window suffices for our prompt-level experiments. Implementing a MEM-style video encoder with tactile fusion is scheduled for v0.3 of the codebase and will enable full-policy training experiments.

Embedder choice matters. The toy `HashingEmbedder` that ships with v0.2 achieves a surprisingly strong Recall@1=75% on the synthetic benchmark, but semantic generalization to paraphrases with distinct lexical forms requires a real encoder. Production deployments should use the `SentenceTransformerEmbedd` we ship or the text encoder of the underlying VLA. We document the substitution clearly.

Rule-based summarization ceiling. Our initial episode summarizer uses deterministic rules to extract narratives and facts. It is adequate for the interface test-bed but lacks the compression and abstraction an LLM-based summarizer would provide. v0.2 will add a batched LLM summarizer that maintains the deterministic fallback for offline or privacy-sensitive settings.

Benchmark coverage. MNEMO-Bench's v1.0 draft selects representative tasks from each of the four source benchmarks, but the exact selection remains calibrated empirically. We will release the task selection criteria and an open leaderboard with the camera-ready.

5.4 Ethical Considerations

Persistent memory raises unique safety concerns: a robot that remembers user interactions across sessions must handle privacy-sensitive information responsibly. Our schema captures facts with explicit provenance and confidence, enabling session-scoped redaction and audit logging. We document a redaction toolkit in the appendix and recommend opt-in memory for consumer deployments.

6 Conclusion

We presented MNEMO-VLA, a unified three-tier memory framework for vision-language-action models. By separating memory along *time scale* (hot/cold/persistent) and *modality* (video/text/retrieval), we expose a coherent abstraction that integrates with existing VLAs via small, documented patches rather than wholesale rewrites.

Our initial release delivers two of the three tiers as a zero-dependency Python package: a semi-structured language memory schema that is both LLM-consumable and programmatically inspectable, and a persistent cross-session store backed by FAISS-ready retrieval. We demonstrate the framework's utility via a Mem-0 integration that requires only two code edits, and we propose MNEMO-Bench, a unified evaluation harness consolidating four existing memory benchmarks. The package is open-sourced under the MIT license, with 20 passing unit tests on CPU and a reproducible quickstart and retrieval benchmark.

We see three directions of future work. First, implementing the hot-memory tier with MEM-style video encoding and tactile fusion will enable end-to-end policy training. Second, the joint training objective proposed in Equation 1 should be validated empirically; preliminary infrastructure is in place. Third, the MNEMO-Bench harness should be released with a public leaderboard to accelerate community adoption of standardized memory evaluation.

Memory has been the quiet bottleneck in robotic manipulation for years; we hope MNEMO-VLA lowers the cost of addressing it.

References

[1] Shuai Bai et al. Qwen3-VL technical report. *arXiv preprint*, 2025.

[2] Kevin Black et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[3] Tianxing Chen, Zanxin Chen, Baijun Chen, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[4] Tianxing Chen, Yuran Wang, Mingleyang Li, et al. RMBench: Memory-dependent robotic manipulation benchmark with insights into policy design. *arXiv preprint arXiv:2603.01229*, 2026.

[5] Egor Cherepanov, Nikita Kachaev, Alexey K. Kovalev, and Aleksandr I. Panov. Memory, benchmark & robots: A benchmark for solving complex tasks with reinforcement learning. *International Conference on Learning Representations (ICLR)*, 2026.

[6] Haoquan Fang, Markus Grotz, Wilbert Pumacay, et al. SAM2Act: Integrating visual foundation model with a memory architecture for robotic manipulation. *International Conference on Machine Learning (ICML)*, 2025.

[7] Songhao Han, Boxiang Qiu, Yue Liao, et al. RoboCerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation. *Conference on Neural Information Processing Systems (NeurIPS)*, 2025.

[8] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[9] Physical Intelligence et al. $\pi_{0.7}$: A steerable generalist robotic foundation model with emergent capabilities. *Tech Report*, 2026.

[10] Moo Jin Kim et al. OpenVLA: An open-source vision-language-action model. *Conference on Robot Learning (CoRL)*, 2024.

[11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[12] Hang Li, Fengyi Shen, Dong Chen, et al. Empowering vision-language-action model with memory via dual-level recurrent queries. *arXiv preprint arXiv:2603.12942*, 2026.

[13] Hao Li, Shuai Yang, Yilun Chen, et al. CronusVLA: Towards efficient and robust manipulation via multi-frame vision-language-action modeling. *arXiv preprint arXiv:2506.19816*, 2025.

[14] Bo Liu, Yifeng Zhu, Chongkai Gao, et al. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.

[15] Songming Liu et al. RDT-1B: A diffusion foundation model for bimanual manipulation. *International Conference on Learning Representations (ICLR)*, 2025.

[16] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, et al. RoboCasa: Large-scale simulation of everyday tasks for generalist robots. *arXiv preprint arXiv:2406.02523*, 2024.

[17] Hao Shi et al. MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. *International Conference on Learning Representations (ICLR)*, 2026.

[18] Ajay Sridhar, Jennifer Pan, Satvik Sharma, and Chelsea Finn. MemER: Scaling up memory for robot control via experience retrieval. *arXiv preprint arXiv:2510.20328*, 2025.

[19] Stone Tao et al. ManiSkill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *arXiv preprint arXiv:2410.00425*, 2025.

[20] Marcel Torne, Karl Pertsch, Homer Walke, et al. MEM: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.

A Reproducibility Checklist

- Code: https://github.com/asimfish/mnemo-vla (MIT)
- Tests: `pytest -q tests/` (20 tests, CPU-only)
- Benchmark: `PYTHONPATH=. python examples/retrieval_benchmark.py`
- Quickstart: `PYTHONPATH=. python examples/quickstart.py`
- Dependencies: Python ≥ 3.10, numpy ≥ 1.24. Optional extras: `faiss-cpu` for scale, `sentence-transformers` for semantic retrieval.
- No GPU required for primitives; hot-memory and VLA training experiments (deferred to v0.4+) will specify their own requirements.

B Language Memory Schema (Formal Grammar)

```
memory      := task_line session_line? subtasks? cross_session? failures?
task_line   := "[TASK] " <string>
session_line:= "[SESSION] " <string>
subtasks    := "[SUBTASKS]" subtask_line+
subtask_line:= integer "." marker description details?
marker      := "[X]" | "[>]" | "[ ]" | "[!]"
details     := "(" <string> ")"
cross_session := "[CROSS-SESSION]" fact_line+
fact_line   := "-" content "(source:" session "," "conf=" float ")"
failures    := "[FAILURES]" failure_line+
failure_line:= "-" description ("->" intervention)?
```

C Example Episode Summaries

Representative cross-session facts produced by the rule-based summarizer include:

- `Task 'Organize kitchen tools.' ended with outcome 'success'.`
- `Knife is in the top-right drawer. (source: 2027-03-10, conf=0.90)`
- `Could not open drawer -> Asked user for help.`